Computer Lab: SQL Server-5

1. Using the SQL Aggregate Functions

As well as retrieving rows and columns from the database, we often want to perform some form of summation or **aggregation** of data. The ISO standard defines five **aggregate functions**:

- COUNT – returns the number of values in a specified column;
- SUM – returns the sum of the values in a specified column;
- AVG – returns the average of the values in a specified column;
- MIN – returns the smallest value in a specified column;
- MAX – returns the largest value in a specified column.

- These functions operate on a single column of a table and return a single value.
- COUNT, MIN, and MAX apply to both numeric and non-numeric fields
- SUM and AVG may be used on numeric fields only.
- Eeach function eliminates (تزيل القيم النل اولا) nulls first and operates only on the remaining non-null values.

COUNT(*) is a special use of COUNT

- It's job to only count all the rows of a table. So if table has 4 rows then the result is 4
- We can use COUNT(*) regardless of whether nulls or duplicate values occur.

If we want to eliminate duplicates before the function is applied, we use the keyword DISTINCT before the column name in the function.

- The ISO standard allows the keyword ALL to be specified if we do not want to eliminate duplicates, although ALL is assumed if nothing is specified.
- DISTINCT has no effect with the MIN and MAX functions.
- However, it may influence the result of SUM or AVG, so consideration must be given to whether duplicates should be included or excluded in the computation.
- DISTINCT can be specified only once in a query.

It is important to note that an aggregate function can be used only in the SELECT list and in the HAVING clause.

- It is incorrect to use it elsewhere.
- If the SELECT list includes an aggregate function and no GROUP BY clause is being used to group data together, then no item in the SELECT list can include any reference to a column unless that column is the argument to an aggregate function.

Staff

staffNO	fName	lName	city	position	salary	branchNo
s1	Gad	Ahmed	Tanta	Manager	60,000	B02
s2	Samy	Tamer	Cairo	Assistant	30,000	B02
s3	Samy	Ahmed	Tanta	Supervisor	50,000	B01
s4	Tamer	Samir	Giza	Manager	40,000	B03

Ex1: Use of COUNT(*)

Find the number of staff with salaries greater than 40,000.

```
SELECT COUNT(*) AS myCount
FROM Staff
WHERE salary > 40,000;
```

- Restricting the query to salary more than 40,000 is achieved using the WHERE clause.
- The total number of staff satisfying this condition can then be found by applying the aggregate function COUNT.
- The result table is shown in Table 1.

Table 1 Result table for Example 1.

myCount
2

Ex2: Use of COUNT (DISTINCT)

How many different positions are there in the Staff table?

```
SELECT COUNT (DISTINCT position) AS myCount
FROM Staff;
```

The result table is shown in Table 2.

Table 2 Result table for Example 2.

myCount
3

Ex3: Use of COUNT and SUM

Find the total number of Managers and the sum of their salaries.

```
SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
WHERE position = 'Manager';
```

- Restricting the query to Managers is achieved using the WHERE clause.

- The number of Managers and the sum of their salaries can be found by applying the COUNT and the SUM functions respectively to this restricted set.
- The result table is shown in Table 3

Table 3 Result table for Example 3.

myCount	mySum
2	100,000

Ex4: Use of MIN, MAX, AVG

Find the minimum, maximum and average staff salary.

SELECT MIN (salary) **AS** myMin, **MAX** (salary) **AS** myMax, **AVG** (salary) **AS** myAvg
FROM Staff;

- In this example we wish to consider all staff and therefore do not require a WHERE clause.
- The required values can be calculated using the MIN, MAX, and AVG functions based on the salary column.
- The result table is shown in Table 4.

Table 4 Result table for Example 4.

myMin	myMax	myAvg
30,000	60,000	45,000

2.Grouping Results (GROUP BY Clause)

- A query that includes the GROUP BY clause is called a **grouped query**.
- It groups the data from the SELECT table(s)
- It produces a single summary row for each group.
- The columns named in the GROUP BY clause are called the **grouping columns**.
- The ISO standard requires the SELECT clause and the GROUP BY clause to be closely integrated.
- When GROUP BY is used, each item in the SELECT list must be **single-valued per group**.
- Further, the SELECT clause may contain only:
  - column names;
  - aggregate functions;
  - constants;
  - an expression involving combinations of the above.

All column names in the SELECT list must appear in the GROUP BY clause unless the name is used only in an aggregate function.

- The contrary is not true: there may be column names in the GROUP BY clause that do not appear in the SELECT list.

- When the WHERE clause is used with GROUP BY, the WHERE clause is applied first, then groups are formed from the remaining rows that satisfy the search condition.
- The ISO standard considers two nulls to be equal for purposes of the GROUP BY clause.
- If two rows have nulls in the same grouping columns and identical values in all the non-null grouping columns, they are combined into the same group.

Ex5: Use of GROUP BY

Find the number of staff working in each branch and the sum of their salaries.

SELECT branchNo, **COUNT**(staffNo) **AS** myCount, **SUM**(salary) **AS** mySum
FROM Staff
GROUP BY branchNo
ORDER BY branchNo;

- It is not necessary to include the column names staffNo and salary in the GROUP BY list because they appear only in the SELECT list within aggregate functions.
- On the other hand, branchNo is not associated with an aggregate function and so must appear in the GROUP BY list.
- The result table is shown in Table 5.

Table 5 Result table for Example 5.

branch No	myCount	mySum
B01	1	50
B02	2	90
B03	1	40

Conceptually, SQL performs the query as follows:

(1) SQL divides the staff into groups according to their respective branch numbers. Within each group, all staff has the same branch number. In this example, we get three groups:
(2) For each group, SQL computes the number of staff members and calculates the sum of the values in the salary column to get the total of their salaries. SQL generates a single summary row in the query result for each group.
(3) Finally, the result is sorted in ascending order of branch number, branchNo.

3. Restricting groupings (HAVING clause)

- The HAVING clause is designed for use with the GROUP BY clause.
- HAVING clause restricts the **groups** that appear in the result table.
- Although similar in syntax, HAVING and WHERE serve different purposes.
  - The WHERE clause filters individual rows going into the result table.
  - HAVING filters **groups** going into the result table.
  - The ISO standard requires that column names used in the HAVING clause must also appear in the GROUP BY list or be contained within an aggregate function.
  - In practice, the search condition in the HAVING clause always includes at least one aggregate function
    - otherwise the search condition could be moved to the WHERE clause and applied to individual rows. (Remember that aggregate functions cannot be used in the WHERE clause.)
  - The HAVING clause is not a necessary part of SQL
  - Any query expressed using a HAVING clause can always be rewritten without the HAVING clause.

Ex6: Use of HAVING

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
HAVING COUNT(staffNo) > 1
ORDER BY branchNo;
```

- This is similar to the previous example with the additional restriction that we want to consider only those groups (that is, branches) with more than one member of staff.
- This restriction applies to the groups and so the HAVING clause is used.
- The result table is shown in Table 6.

Table 6 Result table for Example 6.

branchNo	myCount	mySum
B02	2	90